Domain 2: Tool Design & MCP Integration — Self-Questioning List

CCAR-F Prep — Domain 2 (18% of scored content).

A. Foundations of Tool Design for Claude

1. What is the difference between Claude *requesting* a tool call and Claude *executing* it — who actually decides whether the call runs?
2. Why is this separation between tool selection and tool execution "fundamental to understanding how tool design affects agent reliability"?
3. When a user says "look up this order," how does Claude actually decide which tool to call — by matching a function name, or by matching a description to intent?
4. Why is a tool's description "not documentation for humans but rather a decision signal for Claude"?

How Claude Selects Tools

5. What are the three phases of Claude's tool selection process on each turn?
6. What happens to tool selection reliability when a description is ambiguous — does Claude ask for clarification, or guess?
7. What happens when two tools have similar descriptions — does Claude pick consistently, or alternate unpredictably?
8. If a tool has a misleading name but a clear, accurate description, will Claude still select it correctly — and why does that reveal which signal Claude weighs more heavily?

Three Components of a Tool Definition

9. What are the three components of a tool definition, and which one has "major" impact on reliability versus "minor"?
10. Why does the input schema have "major" impact — what does it specifically determine?
11. If the name has only minor impact on selection reliability, why would a badly chosen name still matter in practice?

Why Tool Design Is an Architectural Concern

12. Why can misrouting, duplicate calls, and cascading errors from bad tool design not be fixed by "no prompt engineering"?
13. How can a single stray keyword in the system prompt "push Claude towards a wrong tool" even when the tool descriptions themselves are fine?
14. When a scenario describes Claude calling the wrong tool, what should be checked *first* — the prompt, the model, the conversation history, or the tool descriptions?

B. Designing Effective Tool Descriptions and Boundaries

15. What four elements does a good tool description need (purpose, use case, parameters, boundaries)?
16. In the weak-vs-strong example ("Search for things" vs. `search_orders`), what specific elements does the strong version add that the weak one lacks?
17. Why does the strong description explicitly say "Do NOT use this for product searches, use search_products instead" — what problem does that sentence prevent?

Preventing Misrouting Between Similar Tools

18. What is the "usual culprit" behind misrouting between two tools?
19. What is the disambiguation pattern, and why must each tool's description reference the other when their purposes are similar but distinct?
20. In the `get_customer_info` vs. `get_customer_orders` example, what specifically does each description need to state about what it does *not* return?

Tool Names vs. Tool Descriptions

21. Why is a tool's name "always secondary" to its description in Claude's selection process?
22. What happens when a name like `process_data` sits next to a description about order lookups — why does this "dissonance" hurt reliability even though the name is theoretically less important?
23. Why should generic names like `helper`, `process`, `handle`, or `execute` be avoided?

Parameter Descriptions

24. What must Claude do when a parameter like `customer_id` or `date_range` has no description at all?
25. Why does an undescribed `status_filter` parameter risk inconsistent inputs across calls?
26. What specifically should a parameter description communicate (format, valid values, constraints like max range)?

Tool Description Anti-Patterns

27. Why does a vague description like "Search for things" fail Claude specifically, not just human readers?
28. What happens when a tool has no boundary statement — what kind of task might Claude wrongly route to it?
29. Why can technical jargon in a description cause Claude to misinterpret domain-specific terms?
30. What confusion results when a description contradicts its own tool's name?
31. When a question describes Claude consistently calling the wrong tool, why is the answer "almost always" to improve tool descriptions rather than add few-shot examples?
32. Why do few-shot examples fix "output format and judgment" but not tool misrouting?

C. Structured Error Responses for Tools

33. Why does a generic error message like "tool failed" give the calling agent "no information to work with"?
34. What four questions can an agent not answer when it receives a generic error (retry? different tool? escalate? tell the user?)?

35. What are the three consequences of a generic error response (blind retry, premature giving up, vague user-facing message)?

The Structured Error Pattern

36. What three fields does a well-designed error response carry, and what does each one let the agent branch on?
37. In the good-vs-bad JSON example, what specifically is missing from the "bad" generic error that the "good" structured error provides?

Error Categories and Recovery Strategies

38. For each error category (`timeout`, `rate_limited`, `permission_denied`, `not_found`, `validation_error`, `service_unavailable`), what is the correct recovery strategy?
39. Why is `permission_denied` never retryable — what does retrying actually accomplish in this case?
40. Why is `not_found` described as "not an error" but rather valid information handled as a valid empty result?
41. Under what condition is a `validation_error` retryable — and when is it not?

Access Failure vs. Valid Empty Result

42. What is the core difference between an access failure and a valid empty result?
43. Why does treating both the same way create "a silent data integrity failure"?
44. For an API returning HTTP 500 versus HTTP 200 with an empty array, what is the correct response category for each?
45. Why does an HTTP 404 "depend on context" rather than always mapping to one category?
46. In the customer-lookup timeout example, what does the "wrong pattern" cause the agent to incorrectly tell the user — and why is that dangerous?

Returning Errors in MCP Tool Implementations

47. How does an MCP tool signal that a call failed, and what does Claude do differently when this flag is absent?
48. What might Claude try to do with error text if `is_error` is not set to `true`?

Error Response Design Principles

49. Why must a category always be included — what does the agent have to do instead if it's missing (parse description text)?
50. Why must retryability always be included — what happens without it (retry everything, or retry nothing)?
51. Why is "Database connection timed out after 30s" considered actionable while "An error occurred" is not?
52. Why is returning an empty result for an access failure called "the most dangerous anti-pattern"?
53. Why does using consistent category names across every tool in a system matter for the coordinator's recovery logic?

D. Tool Distribution Across Agents

54. Why shouldn't a search agent have file-write tools, and why shouldn't a synthesis agent have any search tools at all?
55. What three problems does giving every agent every tool create (decision complexity, inappropriate actions, harder to audit)?
56. Besides being a security measure, why does `allowedTools` also improve tool selection *reliability*?

The Principle of Least Privilege for Tools

57. Why does Claude's decision reliability improve when it has 3 tools to choose from instead of 15?
58. Why are over-permissive tool sets both a reliability risk and a security risk?
59. If a subagent has write access to a database but its job is only to read data, what specific failure mode does restricting it to read-only operations prevent?

Designing Role-Bounded Tool Sets

60. For each role in the reference table (search agent, document analyst, code reviewer, test runner, synthesis agent, customer support agent), what tools are appropriate and which should be excluded?
61. Why should a synthesis agent have no tools at all — what does it work from instead?

allowedTools in the Agent SDK

62. What does the `allowedTools` parameter restrict, and what happens if a search agent is given `allowedTools=["WebSearch", "WebFetch"]` even though its parent coordinator has broader access?

Permission Inheritance in Multi-Agent Systems

63. What happens to a subagent's permissions if the coordinator runs in `bypassPermissions` mode — can this be relaxed per subagent?
64. Why does this inheritance behavior make `allowedTools` "the primary mechanism" for controlling subagent tool scope rather than the permission mode itself?
65. In a scenario where a subagent performed an action outside its intended scope, what is the fix — and why is that fix still necessary even under a restrictive permission mode?

When to Use Tool Restriction vs. When Not To

66. Why should subagents "always" be restricted when the coordinator delegates a specific task?
67. Why might the coordinator itself need broader tool access than its subagents?
68. Why might an interactive Claude Code session justify broader tool access than a CI/CD pipeline invocation?
69. Why should CI/CD pipeline invocations restrict tool access to match the pipeline step's specific purpose?

E. The tool_choice Setting

70. What is the functional difference between `tool_choice` moving Claude from "might call a tool" to "definitely calls this specific tool"?

The Four Modes

71. What is the behavioral difference between `auto`, `any`, `tool`, and `none`?
72. In what scenario is `auto` the right choice — what kind of agent needs the flexibility to sometimes not call a tool?
73. Why is `any` useful when there are multiple extraction schemas for different input types (invoices, receipts, contracts)?
74. Why is `tool` the right choice when "a specific step must always execute" (e.g., metadata extraction first)?
75. When is `none` useful, and what does it prevent Claude from doing for that turn?

tool_choice and Structured Output

76. Why does `tool_choice: auto` risk Claude responding with prose instead of calling an extraction tool?
77. What does setting `tool_choice` to `any` or `tool` guarantee that `auto` does not?
78. What does adding `strict: true` guarantee on top of `tool_choice: tool` — both that the tool is called AND what else?
79. In the invoice-extraction worked example, what specifically breaks the downstream parser when `tool_choice: auto` is used?

Syntax Errors vs. Semantic Errors

80. What is the difference between a syntax error and a semantic error in structured output?
81. Does `strict: true` catch a missing required field — does it catch a value that is technically valid but factually wrong?
82. In the example where `total_amount` is 150.00 but line items sum to 180.00, why does schema enforcement fail to catch this?
83. Why can't schema enforcement catch a customer name that appears in the wrong field, or contradicting fields like a "completed" status with a null completion date?
84. What kind of layer is required to catch semantic errors, since schema enforcement alone cannot?
85. If a question asks how to guarantee both that a tool is always called AND that its inputs match the schema, what combination answers it — and what does a follow-up question about guaranteeing correct *values* require instead?

F. MCP Server Architecture and Integration

86. What does the Model Context Protocol (MCP) allow Claude to do that its built-in tools alone cannot?
87. What three categories of capability does an MCP server expose (tools, resources, prompts), and how does Claude use each differently?
88. What is the functional distinction between "tools perform actions," "resources provide information," and "prompts are workflow shortcuts"?

Project-Level Configuration (.mcp.json)

89. Why is `.mcp.json` the correct scope for tools the entire team needs?
90. What happens to `.mcp.json` when it's committed to version control — do all team members get the same MCP servers?
91. How does `${VENUE_API_KEY}`-style environment variable expansion prevent an API key from ever appearing in version control?

User-Level Configuration (~/.claude.json)

92. What is `~/.claude.json` used for, and why is it not shared with the team?
93. Given the comparison table (Configuration Location, Shared?, Use For), when would a developer use `.mcp.json` versus `~/.claude.json`?

Environment Variable Expansion for Secrets

94. Why would hardcoding secrets directly in `.mcp.json` be a security problem?
95. What does environment variable expansion let each developer do differently while still sharing the same server configuration?

MCP Resources for Cross-Server Efficiency

96. What problem occurs when Claude is connected to multiple MCP servers but has no way to know which server holds a given piece of information?
97. In the "without resources" example, why does Claude end up making three calls with two wasted?
98. How does exposing a resource catalog per server change this to one call with zero wasted, per the Jira/Confluence/Database example?

MCP Prompts as Slash Commands

99. What naming convention do MCP prompts follow when surfaced in Claude Code, and how are they invoked?
100. Why don't MCP prompts auto-load into context, get added to the tool registry, or get surfaced as resources?
101. If a question asks how MCP prompts are accessed in Claude Code, what is the answer?
102. If a question describes Claude making excessive exploratory calls across multiple MCP servers, what is the fix?
103. If a question asks about shared versus personal MCP configuration, which file does each belong in?

G. MCP Error Patterns and Tool Result Design

104. Why is designing the results tools return "as important as" designing the tools themselves?
105. What three failure modes can a poorly designed tool result cause (too verbose, too sparse, error without `is_error` flag)?

The is_error Flag

106. What does setting `is_error: true` in an MCP tool result tell Claude, and how does Claude adapt its behavior in response?

107. If a database query times out and the tool returns a message without `is_error: true`, what might Claude incorrectly try to do with that text?

Designing Effective Tool Results

108. What is the core principle behind designing tool results — "return only what Claude needs," or "return everything the backend produces"?
109. Why does returning 5 relevant order fields instead of 40 raw database columns reduce context consumption?
110. Why does a structured JSON format make it easier for Claude to extract specific values than a prose paragraph would?
111. Why does including status information (e.g., "partial, 3 of 5 records returned") matter for how Claude reasons about the result's completeness?

Verbose vs. Concise Tool Results

112. When is a verbose tool result appropriate, and when does it become a liability?
113. What is the tradeoff risk of a concise tool result — what might it miss?
114. Why is "concise results for production agents" the correct default, with follow-up calls available if more detail is needed?
115. In the 40-field customer lookup example, what is the token cost difference between returning all 40 fields versus filtering to 6 relevant fields across 5 lookups?

When to Transform Tool Results

116. What are the four transformation patterns for tool results (flatten, aggregate, convert codes to labels, add computed fields), and when is each appropriate?
117. Why might transforming a result "too aggressively" become its own problem — what does Claude lose the ability to do?

H. Built-in Tool Selection and Usage Patterns

118. What are the three categories of Claude Code's built-in tools relevant to codebases (file operations, search, execution)?

Built-in Tool Reference

119. For each of Read, Write, Edit, Grep, Glob, and Bash, what is it used for — and what should it explicitly *not* be used for?
120. Why can't Write be used for "targeted changes" — what tool is correct for that instead?

Grep vs. Glob: Content Search vs. Name Search

121. What is the fundamental distinction between Grep and Glob — content versus name?
122. For "find all files that import a specific package," which tool is correct, and why?
123. For "find files named cache-something," which tool is correct, and why?
124. For "find where a distinctive error message is defined across services," which tool applies, and why?

The Read → Write Fallback Pattern

125. Why does the Edit tool require its `old_string` parameter to be unique within the file?
126. What happens when a file has highly repetitive content and Edit cannot find a unique match?
127. What is the reliable fallback pattern when Edit fails on repetitive content (Read → modify in reasoning → Write)?
128. In the 150-line configuration file example, why do the two "wrong approaches" (longer old_string, Bash heredoc append) fail — and why does Read → Write always work regardless of repetition?

Tool Selection for Common Tasks

129. For mapping an unfamiliar codebase's structure, why does the strategy combine Glob first, then Read on key files?
130. For finding all callers of a function, why does the strategy require reading the function's module first before Grep-ing for its exported names?
131. For tracing an error message to its source, why does Grep come before Read in the strategy?
132. For decomposing a large exploration task, why does the strategy sequence Glob → Grep → prioritized plan rather than reading everything directly?

Agent SDK Built-in Tools Beyond File Operations

133. What is ToolSearch, and what specific context-consumption problem does it solve?
134. Why does loading all possible tool definitions at session start "consume context with tool definitions" even before any tool is used?
135. If a question describes an agent with many tools where tool definitions consume too much context, what is the mechanism that fixes it?
136. If a question asks about the mechanism for spawning subagents specifically (as opposed to loading tools on demand), which built-in tool answers it?

Cross-Cutting Self-Questioning Prompts for Domain 2

137. For any scenario describing Claude calling the wrong tool repeatedly — is the root cause the prompt, the model, or (almost always) the tool descriptions themselves?
138. For any scenario describing a coordinator that "cannot tell what went wrong" from a tool failure — is the fix a structured error response with category, description, and retryability?
139. For any scenario describing a downstream agent reporting "no data" when the real problem was an unreachable source — is this an access-failure-vs-valid-empty-result confusion?
140. For any scenario describing a subagent acting outside its intended scope — is `allowedTools` the fix, even if the coordinator itself runs in a permissive mode?
141. For any scenario needing both guaranteed tool invocation and guaranteed input structure — does the answer combine `tool_choice` with `strict: true`, and does a follow-up about correct *values* signal that a separate validation layer is needed?
142. For any scenario describing excessive exploratory calls across multiple MCP servers — is the fix MCP resources (content catalogs), not additional tools?
143. For any scenario describing context consumed by tool definitions before the first user message — is ToolSearch the on-demand loading mechanism being tested?

144. For any scenario contrasting "searching by content" versus "searching by name" in a codebase — is the distinction actually testing Grep vs. Glob?